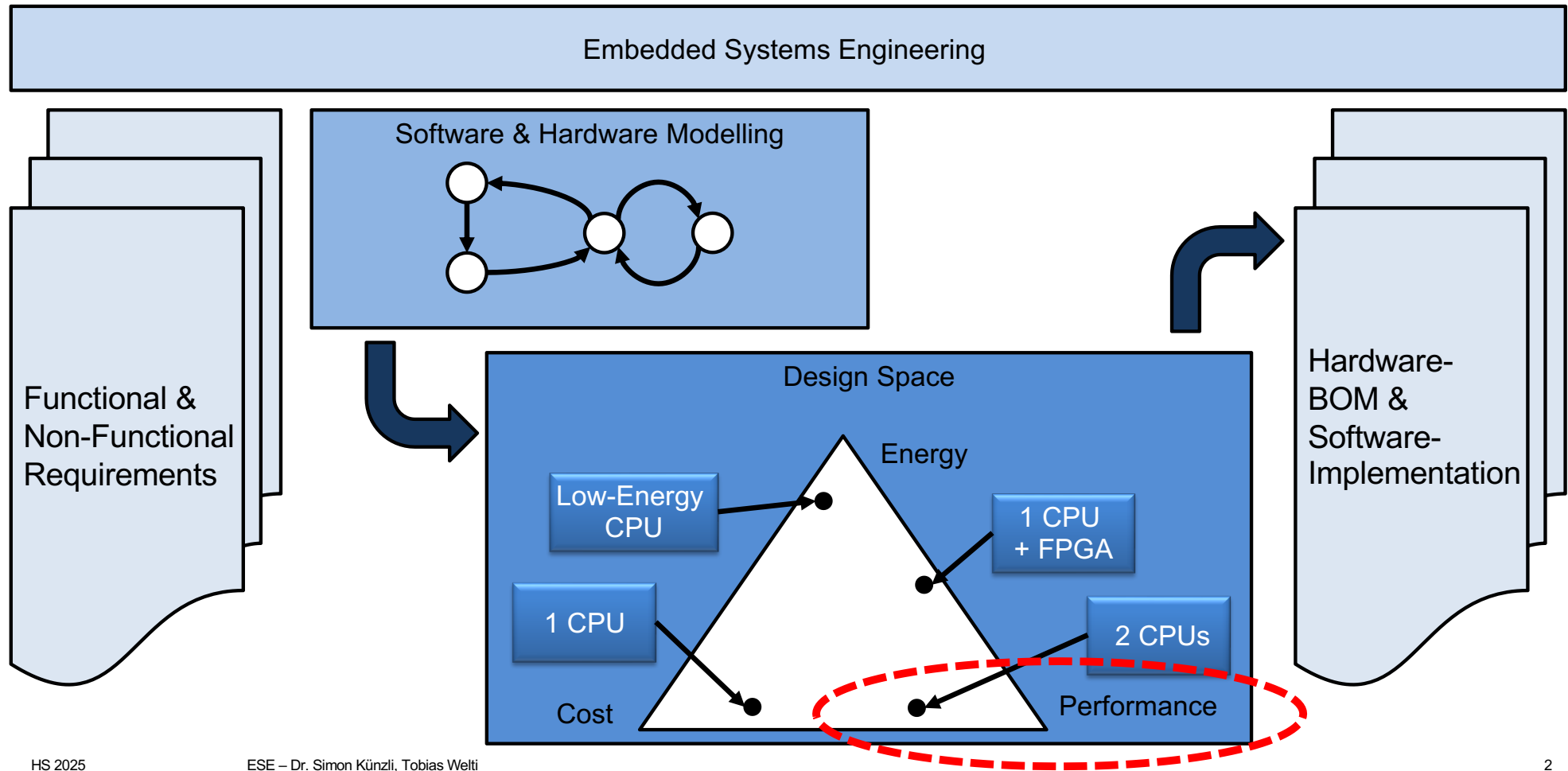


Embedded Software Engineering Performance and Optimization

Simon Künzli, Tobias Welti
HS 2025

Advance Organizer ESE



Goals

After today's lesson, you should be able to ...

- ... use performance metrics to decide which part of your software should be optimized
- ... understand the principal operation of the performance monitor unit of the Cortex-A53
- ... discuss software and hardware optimization techniques
- ... decide which optimization techniques to apply depending on your algorithm
- ... understand the effect of optimization flags for your compiler
- ... understand the principle of vectorized processing (SIMD)

Performance?

■ What performance metrics do you know?

-
-
-
-

■ How would you improve performance?

-
-
-
-

Take the right steps

1. **Set clear performance goals**
2. **Identify bottlenecks**
3. **Profile the code**
4. **Optimize algorithms and data structure**
5. **Use the caches**
6. **Use parallelization, concurrency**

1. Performance goals

■ Execution time

- Meeting deadlines & requirements
- Return to low-power mode sooner

■ Response Time (Latency)

- User experience? Hard time limits?

■ Memory footprint

- Code size, Data size: the program and data must fit into the microcontroller
- Work inside the working set (cache)

■ CPU utilization

- Feasible schedule? Interrupt service time?
- Keep the utilization below $x\%$

2. Identify bottlenecks

Common bottlenecks:

- **Bad code**
 - Polling vs. interrupts
- **Inefficient algorithms**
 - Memory thrashing
 - Subsequent for loops
- **Lack of hardware resources**
 - Too few peripherals, need to multiplex interfaces
 - Too little memory
 - Small cache
- **Bad architectural decisions**
 - Using the wrong interfaces or protocols
 - Inefficient data structures

- **Using Hardware: Segger J-Trace, Lauterbach Trace32**

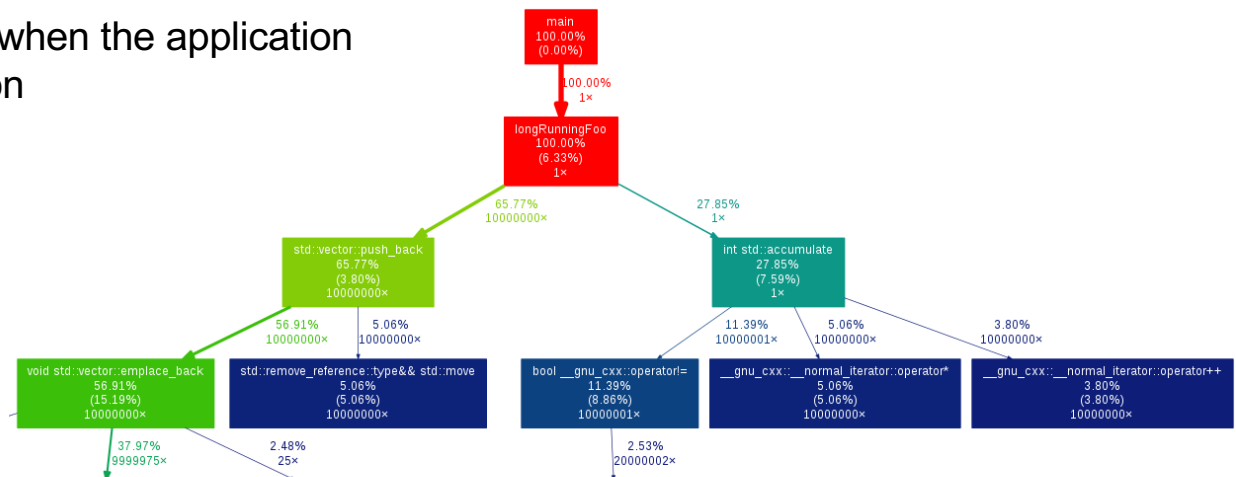
- [illegible]

Flame graph:

-
- time
- Visualize Total Time

3. Profiling Tools

- **Using Software: gprof, Oprofile, valgrind**
 - Function-level profiling
 - Create a function call-graph
 - Introduces a software overhead but identifies the most resource-hungry functions
 - Identify memory leaks, cache thrashing
- **Without any OS or debugging interface: Profiling by “printf” or I/O:**
 - Use GPIO pins to indicate when the application enters and leaves a function



3. Profiling: Performance Monitor Unit (PMU)

ARM Cortex-A53 has a built-in performance monitor unit

- Use like a peripheral
- Configuration via Registers
- Six counters for events in the CPU
- Countable Events:
 - Bus cycles,
 - Bus accesses,
 - Cache-related events: hits, misses, allocations
 - memory accesses,
 - CPU cycles,
 - load and store instructions
 - branches taken, pipeline flushes,
 - branches predicted correctly...

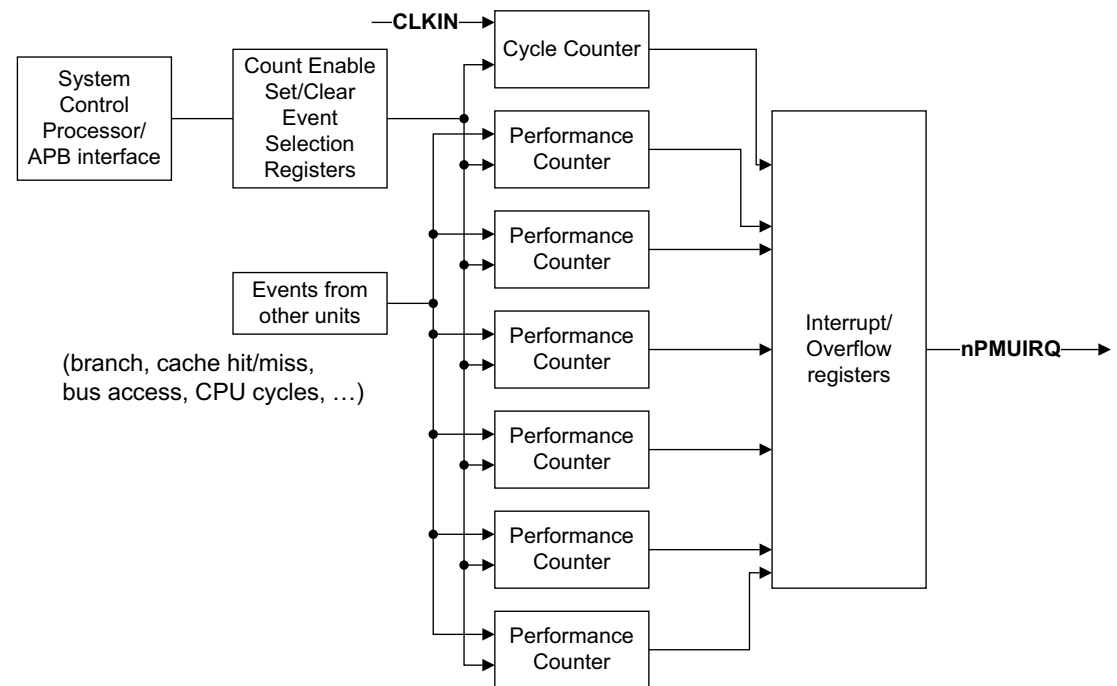


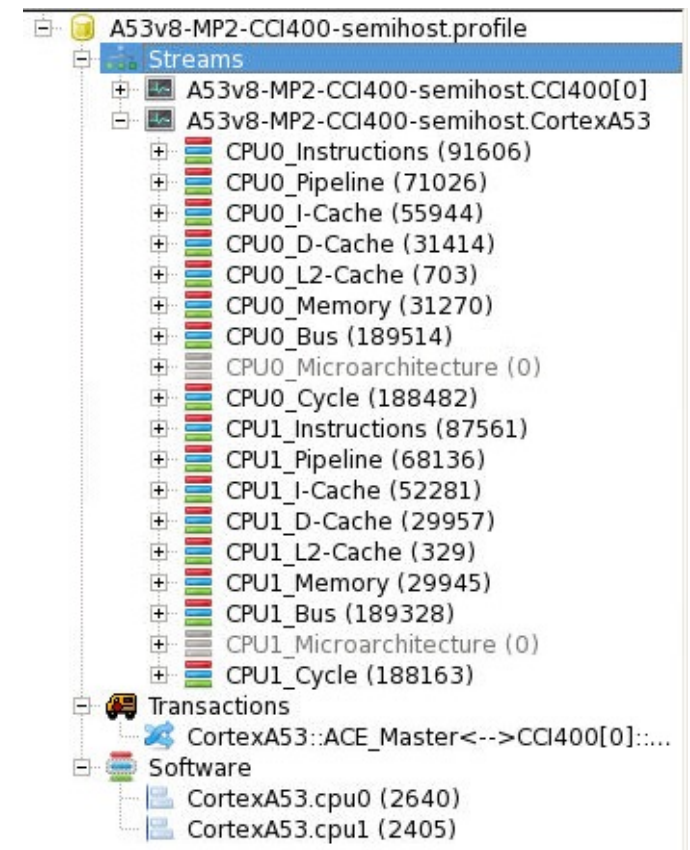
Figure 12-1 PMU block diagram

ARM Cortex-A53 Technical Reference Manual, DDI0500D

3. Profiling: Performance Monitor Unit (PMU)

Using the PMU

- set the type of events to count
- select which counters to use
- enable counters
- execute your software (or the section of interest)
- read out counters
- For details, see:
ARM Cortex-A53 Tech. Ref. Manual Chapter 12
- or: use the Tools, i.e., **Carbon Analyzer GUI**



www.arm.com, Carbon Analyzer GUI

4. Optimize algorithms and data structure

■ Avoid unnecessary loading and storing of data

- Data transfer costs energy and processor cycles
- Once you load data, perform all operations on it. Then, store the results.
- Compiler mostly takes care of this, but you can help:
 - two subsequent for loops over the same index and/or data range should be combined

■ Profile and benchmark library functions

- Someone might have found a better solution already

■ Data structure

- Structs may be convenient, but...
- Multiple Arrays can make better use of cache: i.e. you only need `ch_a[]`:

Array of struct:

ch_a[0]	ch_b[0]	ch_c[0]	ch_a[1]	ch_b[1]	ch_c[1]	ch_a[2]	ch_b[2]
---------	---------	---------	---------	---------	---------	---------	---------

Arrays:

ch_a[0]	ch_a[1]	ch_a[2]	ch_a[3]	ch_a[4]	ch_a[5]	ch_a[6]	ch_a[7]	...
ch_b[0]	ch_b[1]	ch_b[2]	ch_b[3]	ch_b[4]	ch_b[5]	ch_b[6]	ch_b[7]	...
ch_c[0]	ch_c[0]	ch_c[0]	ch_c[0]	ch_c[0]	ch_c[0]	ch_c[0]	ch_c[7]	...

5. Use the caches

■ Know your hardware

- Respect cache sizes and use the cache as much as possible.

■ Multitasking can thwart your caching strategy

- Context switches do not include cache contents
- L1 cache most likely gets replaced, but L2 might keep your working set

```
struct multichannel_t{
    uint16_t ch_a;
    uint16_t ch_b;
    uint16_t ch_c;
} multichannel_t;

int main(void) {
    struct multichannel_t input[128] = {0};
```

cache lines for access to ch_a[] and ch_b[]:

Array of struct:

ch_a[0]	ch_b[0]	ch_c[0]	ch_a[1]	ch_b[1]	ch_c[1]	ch_a[2]	ch_b[2]
---------	---------	---------	---------	---------	---------	---------	---------

```
struct multichannel_array_t{
    uint16_t ch_a[128];
    uint16_t ch_b[128];
    uint16_t ch_c[128];
} multichannel_array_t;
```

Arrays:

ch_a[0]	ch_a[1]	ch_a[2]	ch_a[3]	ch_a[4]	ch_a[5]	ch_a[6]	ch_a[7]
ch_b[0]	ch_b[1]	ch_b[2]	ch_b[3]	ch_b[4]	ch_b[5]	ch_b[6]	ch_b[7]
ch_a[8]	ch_a[9]	ch_a[10]	ch_a[11]	ch_a[12]	ch_a[13]	ch_a[14]	ch_a[15]

6. Use parallelization, concurrency

If you need more computing power or shorter execution times:

■ **Use parallelization**

- Vector instructions execute an operation on multiple data sets.
- Event-driven programming is a form of parallelization.

■ **Asynchronous programming**

- Use interrupts to execute functions only when required.

■ **An RTOS can help:**

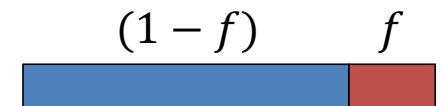
- Multiprocessing (multiple tasks or threads)
- Priorities allow light-weight periodic tasks to preempt heavy loads.

Amdahl's Law

How much will overall performance improve?

- Amdahl's Law calculates the overall speedup $S_{overall}$ when a portion f of a program is improved by a factor of a .

$$S_{overall} = \frac{T_{orig}}{\left[(1 - f) + \frac{f}{a} \right] * T_{orig}} = \frac{1}{(1 - f) + \frac{f}{a}}$$



- f : fraction of the **runtime** used on the improved section
- a : runtime improvement factor
- T_{orig} : runtime of the whole program before improvement

improvement: speedup in percent

$$improvement = 100\% \left(1 - \frac{1}{S_{overall}} \right)$$

Amdahl's Law

Example:

- Improve a section of the program that consumes **10 %** of total runtime by a factor of **10'000**.

$$S_{overall} = \frac{T_{orig}}{\left[(1 - f) + \frac{f}{a} \right] * T_{orig}} = \frac{1}{(1 - f) + \frac{f}{a}} =$$

- How much overall speedup do we get? And in percent?

$$improvement = 100\% \left(1 - \frac{1}{S_{overall}} \right) =$$

Amdahl's Law

Example:

- Improve a section of the program that consumes **90 %** of total runtime by a factor of **100**.

$$S_{overall} = \frac{T_{orig}}{\left[(1 - f) + \frac{f}{a} \right] * T_{orig}} = \frac{1}{(1 - f) + \frac{f}{a}} =$$

- How much overall speedup do we get? And in percent?

$$improvement = 100\% \left(1 - \frac{1}{S_{overall}} \right) =$$

Amdahl's Law

Limits of Amdahl's Law:

- With $f \rightarrow 1$, $S_{overall} = a$
- With $a \rightarrow \infty$, $S_{overall} = 1/(1 - f)$

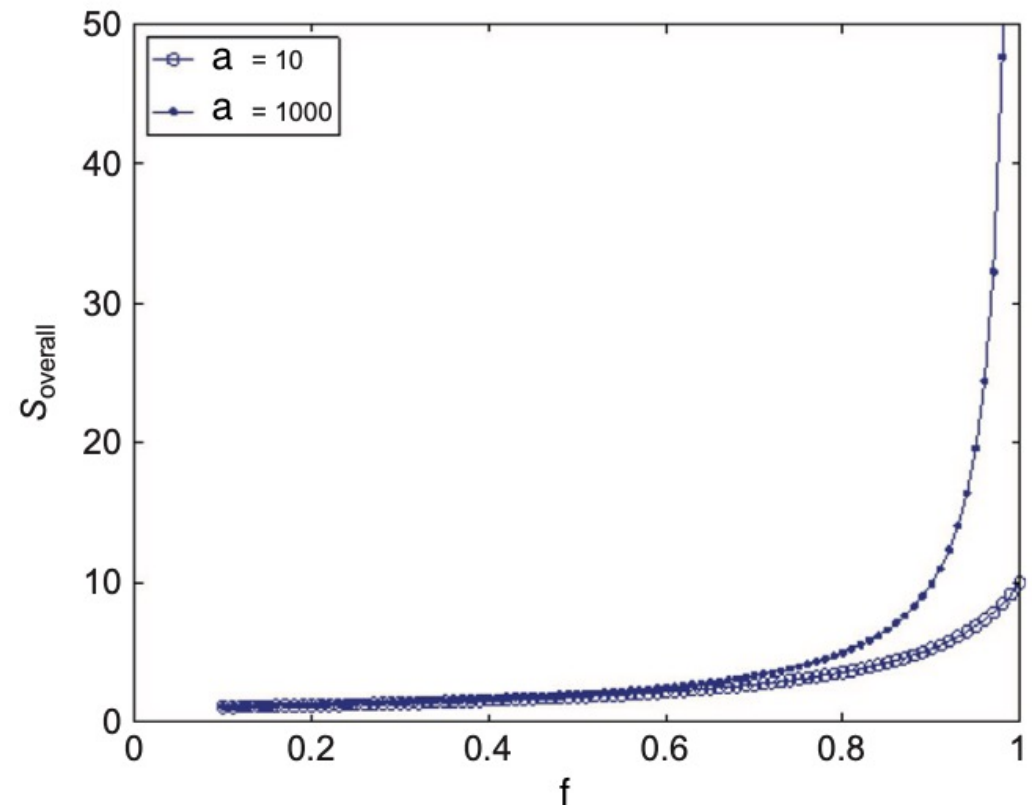
Application tips:

- *Profile! Don't speculate.*
Measure the performance of your software to identify potential sections to be accelerated.
- Balance development effort for acceleration according to how much can be gained.

Amdahl's Law

Difficult to reach a high $S_{overall}$:

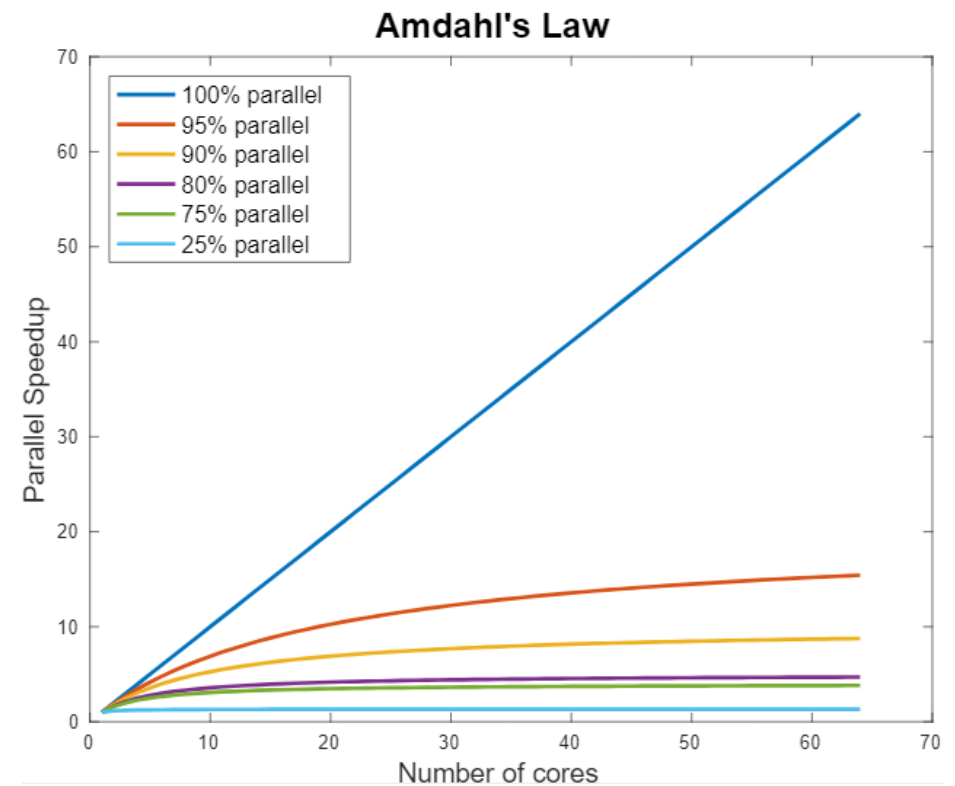
- only for large f can there be a significant improvement
- Acceleration factor a plays a secondary role
 - $f = 97\% \rightarrow$ factor of 4.1x between curves for $a = 10$ or 1000
 - speedup is 250x less than $a = 1000$
- **→ balance your effort!**



Amdahl's Law: Limits

Amdahl's law limits the scaling

- Better parallelization makes multicore execution scale better.
- But: Inter-processor communication takes time.
 - data exchange via memory
- Large number of cores for single task only works for large datasets.



Consider data structure

Recap CT

Respect spatial and temporal locality

- use what's in the Cache
- process all that's in the Cache

```
j is inner
1047527424-1048575999 done

real    0m3.460s
user    0m3.452s
sys     0m0.000s

j is outer
1047527424-1048575999 done

real    0m18.509s
user    0m18.472s
sys     0m0.000s
```

```
if (argc==2) {
    printf("j is inner\n");
    for (int n=0; n<iterations; n++) {
        for (int i=0; i<size; i++) {
            for (int j=0; j<size; j++) {
                array[i][j] = v;
                v++;
            }
        }
    }
} else {
    printf("j is outer\n");
    for (int n=0; n<iterations; n++) {
        for (int j=0; j<size; j++) {
            for (int i=0; i<size; i++) {
                array[i][j] = v;
                v++;
            }
        }
    }
}
```

Optimizing SW Execution Time: Algorithm

Choose an efficient algorithm

Example: fourth-degree polynomial calculation with float values.

Naïve approach: How many multiplications/additions?

$$y(x) = a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0$$

Optimizing SW Execution Time: Algorithm

■ Horner's rule for polynomial evaluation

$$y(x) = a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0$$

$$d_0 = a_4$$

$$d_1 = x \cdot d_0 + a_3 = a_4x + a_3$$

$$d_2 = x \cdot d_1 + a_2 = a_4x^2 + a_3x + a_2$$

$$d_3 = x \cdot d_2 + a_1 = a_4x^3 + a_3x^2 + a_2x + a_1$$

$$d_4 = x \cdot d_3 + a_0 = a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0$$

Total:

vs 10 multiplications and 4 additions

Optimizing SW Execution Time: Loop Unrolling

Loop unrolling

- Program spends a significant portion of execution time in loops
 - loops are often worth checking for optimization potential
- Small loops can be unrolled
 - skip the condition checks → less branches
 - update loop counter less often
 - increases code size

Optimizing SW Execution Time: Loop Unrolling

Loop unrolling example: counting '1'-bits in a value

```
int countbit1(unsigned int n)
{
    int bits = 0;
    while (n != 0)
    {
        if (n & 1) bits++;
        n >>= 1;
    }
    return bits;
}
```

```
int countbit2(unsigned int n)
{
    int bits = 0;
    while (n != 0)
    {
        if (n & 1) bits++;
        if (n & 2) bits++;
        if (n & 4) bits++;
        if (n & 8) bits++;
        n >>= 4;
    }
    return bits;
}
```

Optimizing SW Execution Time: Loop Unrolling

Loop unrolling example:

- Unrolled code size is significantly larger
- Faster execution with fewer condition checks and fewer branches
- Let the compiler do it for you:
-O3

```
countbit1:
    mov     r1, r0
    mov     r0, #0
    cmp     r1, #0
    bxeq    lr
    mov     r2, #0
.LBB0_1:
    and     r3, r1, #1
    cmp     r2, r1, lsr #1
    add     r0, r0, r3
    lsr     r3, r1, #1
    mov     r1, r3
    bne     .LBB0_1
    bx      lr
```

```
countbit2:
    mov     r1, r0
    mov     r0, #0
    cmp     r1, #0
    bxeq    lr
    mov     r2, #0
.LBB1_1:
    and     r3, r1, #1
    cmp     r2, r1, lsr #4
    add     r0, r0, r3
    ubfx    r3, r1, #1, #1
    add     r0, r0, r3
    ubfx    r3, r1, #2, #1
    add     r0, r0, r3
    ubfx    r3, r1, #3, #1
    add     r0, r0, r3
    lsr     r3, r1, #4
    mov     r1, r3
    bne     .LBB1_1
    bx      lr
```

Optimizing SW Execution Time: Conditions

Condition optimization

- Boolean expression evaluation terminates as soon as the result is defined.
- $A \ || \ B$
 - if A evaluates true, B will not need to be executed
 - put the condition with the highest probability to be true first
- $A \ \&\& \ B$
 - if A evaluates false, B will not be executed
 - put the condition with the highest probability to be false first

Optimizing SW Execution Time: Loops

Loop performance recap

- Use simple termination conditions
 - condition must be evaluated every iteration
 - avoid function calls in condition
- Optimize order of conditions in Boolean expressions
 - OR: probably true condition first
 - AND: probably false condition first
- Unroll small loops

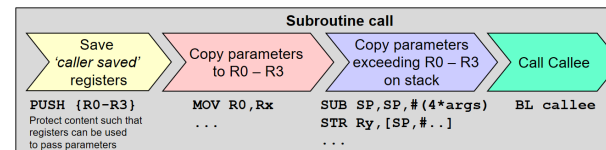
Subroutine calls

Recap CT

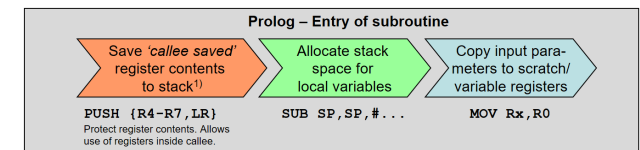
Subroutine calls overhead:

- Parameter passing
 - Parameters in registers are cheapest
 - four parameters on ARM Cortex-R5, eight on A53
- Stack frame
 - Storing/restoring registers that need to be preserved
 - Link Register, extra parameters
- Subroutines should be worth the overhead
 - do a significant amount of work

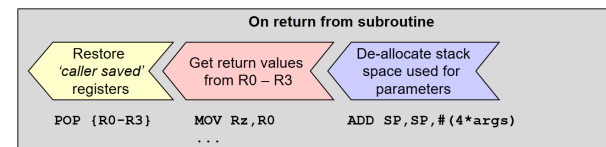
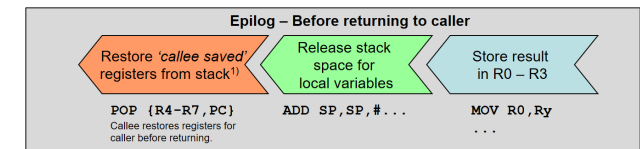
Caller



Callee



Code of subroutine



Optimizing SW Execution Time: Inline Functions

Avoiding subroutine calls:

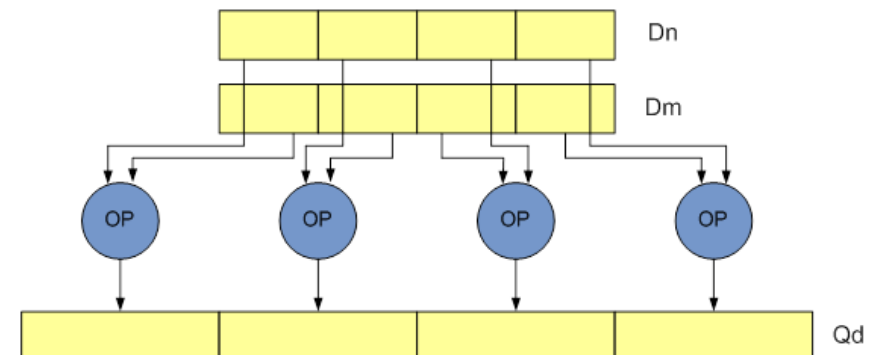
- Inline functions replace the subroutine call with the subroutine code.
- Inline functions increase the code size.
- The compiler usually decides for itself whether to inline code or not.

```
inline int add(int i, int j) {  
    return i + j;  
}  
  
int main() {  
    int i = add(4, 5);  
    return i;  
}
```

Vector processing

ARM processors can process data in vectors

- ARM NEON extends the instruction set architecture with Single Instruction, Multiple Data Instructions (SIMD).
- SIMD executes one instruction on a vector of data
- Example: four multiplications in one step
four 32-bit values D_n *
four 32-bit values D_m =
four 64-bit values Q_d
- A53 has an extra set of 32x 128-bit registers designed for NEON instructions
- 16x int8_t,
8x int16_t,
2x double, ...



Vector processing

Vector processing on ARM

- NEON instructions are available in ARM Assembly and can be used as *intrinsic* directly in C code.
 - Intrinsics are function calls that are replaced with NEON assembly instructions
- Coding NEON in Assembly gets dirty quickly... not covered in this course.
- ➔ let the compiler do it for you!

2.1.1.2.5. Fused multiply-accumulate ^A

Intrinsic	Argument preparation	AArch64 Instruction	Result	Supported architectures
float32x2_t <code>vfma_f32</code> (float32x2_t a, float32x2_t b, float32x2_t c)	a -> Vd.2S b -> Vn.2S c -> Vm.2S	FMLA Vd.2S,Vn.2S,Vm.2S	Vd.2S -> result	v7/A32/A64
float32x4_t <code>vfmaq_f32</code> (float32x4_t a, float32x4_t b, float32x4_t c)	a -> Vd.4S b -> Vn.4S c -> Vm.4S	FMLA Vd.4S,Vn.4S,Vm.4S	Vd.4S -> result	v7/A32/A64
float64x1_t <code>vfma_f64</code> (float64x1_t a, float64x1_t b, float64x1_t c)	b -> Dn c -> Dm a -> Da	FMADD Dd,Dn,Dm,Da	Dd -> result	A64

Vector processing

Auto-vectorizing

- The compiler can recognize sections that can be vectorized.
- Optimization levels `-O2`, `-O3` perform auto-vectorizing
- Vectorizing can help reduce code size
 - Unrolled loops converted to vector instructions
 - Further optimization options: `-Os` (reduced code size), `-Oz` (smallest code size)

Vector processing

Let the compiler vectorize

```
void vec_add(float *vec_A, float *vec_B, float *vec_C, int len_vec) {
    int i;
    for (i=0; i<len_vec; i++) {
        vec_C[i] = vec_A[i] + vec_B[i];
    }
}
```

unvectorized

```
vec_add ; Alternate
    CMP     w3,#1
    B.LT    |L3.36|
    MOV     w8,w3
    |L3.12|
    LDR     s0,[x0],#4
    LDR     s1,[x1],#4
    SUBS    x8,x8,#1
    FADD    s0,s0,s1
    STR     s0,[x2],#4
    B.NE    |L3.12|
```

vectorized

```
|L3.108|
    LDR     q0,[x13],#0x10
    LDR     q1,[x12],#0x10
    SUBS    x10,x10,#4
    FADD    v0.4S,v0.4S,v1.4S
    STR     q0,[x11],#0x10
    B.NE    |L3.108|
    CMP     x9,x8
    B.EQ    |L3.184|
```

preparation

```
vec_add ; Alternate
    CMP     w3,#1
    B.LT    |L3.184|
    CMP     w3,#4
    MOV     w8,w3
    MOV     x9,xzr
    B.CC    |L3.140|
    LSL     x10,x8,#2
    ADD     x12,x0,x10
    ADD     x11,x2,x10
    CMP     x12,x2
    ADD     x10,x1,x10
    CSET    w12,HI
    CMP     x11,x0
    CSET    w13,HI
    CMP     x10,x2
    CSET    w10,HI
    CMP     x11,x1
    AND     w12,w12,w13
    CSET    w11,HI
    TBNZ    w12,#0,|L3.140|
    AND     w10,w10,w11
    TBNZ    w10,#0,|L3.140|
    AND     x9,x8,#0xffffffffc
    MOV     x10,x9
    MOV     x11,x2
    MOV     x12,x1
    MOV     x13,x0
```

Vector processing

Vectorizing pros and cons

- Process multiple data (2, 4, 8, 16, depending on data type) in a single instruction.
- Code size increases:
 - preparing vector registers has an overhead (separate pipeline)
 - handling of vector sizes that are not a multiple of the register width (zero padding, skip results)
- This is only possible if no dependencies between loop iterations exist.
- No complex loops conditions allowed

Vector processing

Helping the compiler

Hints can help the compiler reduce the vectorization overhead:

■ **Pragmas:** `#pragma clang loop vectorize(enable) vectorize_width(2)`

- "you can process two values at a time"
- pragmas not covered in this course

■ **Tell the compiler that a vector will always be a multiple of four:**

- `for (i=0; i<(len_vec & ~3); i++)`
- "There is no need to check the vector size"
- helps reduce code size and execution time

```
void vec_cubed(double *restrict x_vec, double *res  
    int i;  
#pragma clang loop vectorize(enable) vectorize_width(fi  
    for (i=0; i<(len_vec & ~3); i++) {  
        y_vec[i] = cubed_i(x_vec[i]);  
    }  
}
```

Examples

How would you optimize?

```
int main(void) {  
    int fact[5];  
    fact[0] = 1;  
  
    for (int i = 1; i < 5; ++i) {  
        fact[i] = fact[i - 1] * i;  
    }  
}
```

Examples

How would you optimize?

```
#include <stdio.h>
int main(void)
{
    int arr[1000];
    int a = 1, b = 5, c = 25, d = 7;

    for (int i = 0; i < 1000; ++i) {
        arr[i] = (((c % d) * a / b) % d) * i;
    }
    return 0;
}
```

Examples

How much speedup?

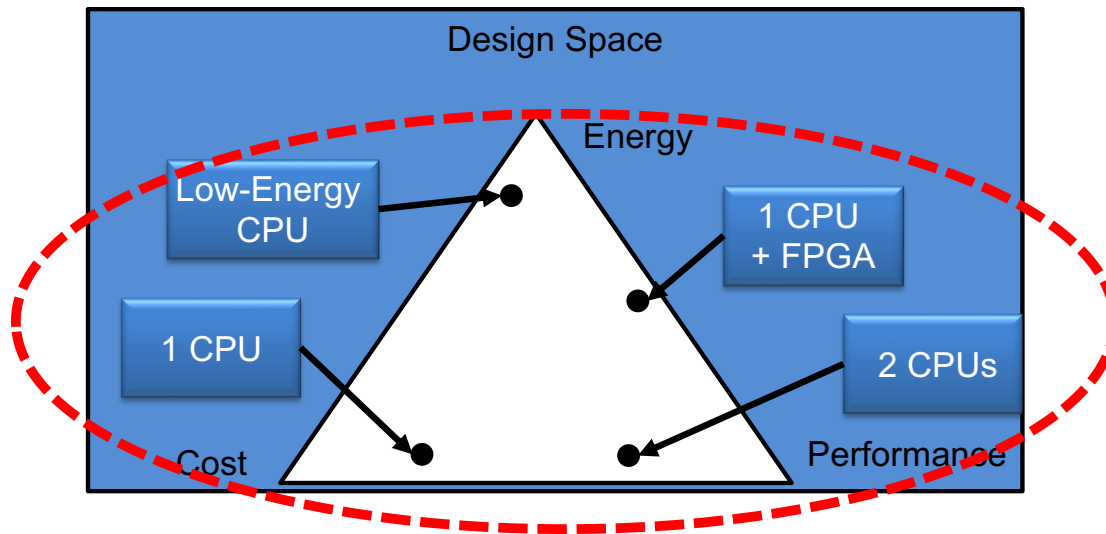
- for loop took 98 % of the execution time
- for loop executes 3000 times as fast after optimization
- What value for $S_{overall}$?
- How many percent improvement?

```
int main(void)
{
    int arr[1000];
    int a = 1, b = 5, c = 25, d = 7;

    // pre calculating the constant expression
    int temp = (((c % d) * a / b) % d);

    for (int i = 1000; i != 0; --i) {
        arr[i] = temp * i;
    }
}
```


Evaluation of optimization



- **Trading, balancing**
 - performance vs
 - code size vs
 - memory footprint vs
 - microcontroller complexity
- **Memory, HW cost**
- **Development cost**
- **Requirements met?**
- **Specialized HW?**

Summary

- **Set the goal.**
- **Identify significant potential for software improvement.**
 - Amdahl's Law → Profile! Don't speculate.
- **Use SIMD instructions**
 - Let the compiler do it
 - Help the compiler see optimization potential: compiler hints
- **Trade code size vs performance:**
 - loop unrolling, loop optimizations
 - inline functions
- **Remember CT: Cache size, data arrangement in memory**
- **If an algorithm needs more computing power, Can you parallelize it?**
→ Consider FPGA or more cores